XSV [This version is still work in progress! Typing rules are not finished and still contains errors. (05 June 2024)]

On the design and implementation of Modular Explicits

SAMUEL VIVIEN, OCamlPro, France and École Normale Supérieure, PSL Research University, France
DIDIER RÉMY, Cambium, INRIA, France

We present and discuss the design and implementation of *modular explicits*, an extension of OCaml first-class modules with *module-dependent functions*, functions taking first-class modules as arguments. We show some difficulties with the present use of first-class modules and how modular explicits solve them in a simpler, more direct way. Modular explicits are fully compatible with, and can be presented as an extension of, first-class modules. Interestingly, both the formalization and the implementation reuse the mechanism designed to ensure principal types in the presence of semi-explicit first-class polymorphism and OCaml polymorphic methods [2]. Modular explicits are also meant to be the underlying language in which *modular implicits* [4], i.e., module arguments left implicit from their signatures, should be elaborated.

ACM Reference Format:
Samuel Vivien and Didier Rémy. 2024. On the design and implementation of Modular Explicits. 1, 1 (June 2024), 10 pages.

1 MOTIVATION

First-class modules in OCaml

First-class modules, a feature present in OCaml since 3.12, allow a module M of signature S to be packed as a value m of type $(\mathbf{module}\ S)$ using the construct $(\mathbf{module}\ M : S)$, which can then be stored in data-structures or passed as argument to functions. A first-class module such as m, i.e., a value of *known* type $(\mathbf{module}\ S)$ can then be unpacked with the construct

$$\mathbf{let\ module}\ \mathcal{X} = (\mathbf{val}\ m)\ \mathbf{in}\ a[X]$$

This bind $\mathcal{X}$ to a module of type S that can be used in the expression a. If m were of an unknown type, we should write $(m : (\mathbf{module}\ S))$ instead of m. This situation occurs in particular when x is a function argument, as in

$$\lambda x.\,\mathbf{let\ module}\ \mathcal{X} = (\mathbf{val}\ (x : (\mathbf{module}\ S)))\ \mathbf{in}\ a[X] \tag{1}$$

or equivalently, moving the annotation to the binding site,

$$\lambda x : (\mathbf{module}\ S).\,\mathbf{let\ module}\ \mathcal{X} = (\mathbf{val}\ x)\ \mathbf{in}\ a[X]$$

Here, the first-class module is immediately unpacked as a module $\mathcal{X}$ which can then be used in $a[\mathcal{X}]$. As this is a common pattern, OCaml already offers the following concise abbreviation:

$$\lambda(\mathbf{module}\ \mathcal{X} : S).\,a[\mathcal{X}]$$

giving the (useful) illusion that the function directly receives a module as argument.

Unfortunately, there is some limitation to this pattern when the signature S contains an abstract type component, say $\mathbf{type}\ t$. Looking at the equivalent form (1), it is clear that the first-class module type $(\mathbf{module}\ S)$ of x contains an abstract type t, which when unpacked becomes the abstract type $\mathcal{X}.t$ with a scope limited to the let-binding body, i.e., $a[\mathcal{X}]$. That is, $\mathcal{X}.t$ should not occur free in the type of $a[\mathcal{X}]$, which is also the type of the let-binding as it would otherwise escape from its scope.

Authors' addresses: Samuel Vivien, OCamlPro, France and École Normale Supérieure, PSL Research University, France; Didier Rémy, Cambium, INRIA, France.

2024. XXXX-XXXX/2024/6-ART $15.00
https://doi.org/

A possible work around?

When we write $\lambda(\textbf{module}\ \mathcal{X} : S).\, a$ in such a case, we rather mean a polymorphic function of type $\forall\alpha.\, (\textbf{module}\ S\ \textbf{with type}\ t = \alpha) \to \tau[\alpha]$, say σ, That is,

$$\lambda(\textbf{type}\ \alpha).\, \lambda(\textbf{module}\ \mathcal{X} : S\ \textbf{with type}\ t = \alpha).\, a[\mathcal{X}]$$

which accepts as argument any module compatible with the signature S, i.e., with a type component t that might be abstract or any concrete type, much as during the application of a functor $\lambda(\mathcal{X} : S).\, M'$. Using an universal binder that extends to the whole arrow type, instead of an existential limited to the right hand side, avoids the scope escaping problem.

Notice however that this amounts to re-encoding the convenient module-level type abstraction mechanism into the core language, somewhat along the lines of [1], hence loosing the convenient and concise path-based approach of OCaml modules. In particular, it does not scale well when S has multiple abstract types.

Besides, we still have to deal with first-class polymorphism when passing such a function, say f, to another higher-order function, say *happy*, that will apply f several times to module arguments of different types:

$$\textbf{let}\ \mathit{happy}\ (f : \sigma) = \ \ldots\ f(\textbf{module}\ M_1)\ \ \ldots\ f(\textbf{module}\ M_2)\ \ \ldots$$

a pattern that often happens in some use cases of modular explicits, such as the emulation of overloading.

Unfortunately, this is not allowed in OCaml when σ is polymorphic as is the case here. OCaml offers some but still poor support for first-class polymorphism. We need to encapsulate σ, either using semi-explicit polymorphism [2] via objects or records polymorphic fields or using some module-level wrapper. Examples of such encodings are given in §B.1.

In fact, this encoding of abstract types with universal types, which may be a work around in some cases, is not possible in OCaml when t is a higher-order abstract type, such as List.t, since OCaml core-level type variables cannot be of higher-rank.

A correct, but unpractical encoding

An alternative solution is to view and encode $\lambda(\textbf{module}\ \mathcal{X} : S).\, a$ altogether as a first-class functor

$$(\textbf{module}\ \lambda(\mathcal{X} : S).\, \textbf{struct}\ \mathit{value} = a\ \textbf{end} : S')$$

whose signature S', equal to $\mathcal{X}.t \to \textbf{sig val}\ \mathit{value} : \tau[\mathcal{X}.t]\ \textbf{end}$, should be *named* and *explicitly* given[1] to build the first-class value (or when passing it to a function such as *happy*). See an example of this encoding in Figure 3.

This encoding can cope with higher-order abstract types and seems to cover all use cases, showing that modular explicits do not provide additional expressiveness—if we ignore conciseness and the amount of type annotations. Indeed, it requires quite a few module-types manipulations, which may quickly become cumbersome, as module types cannot be passed directly to functors in OCaml but only when embedded in structures—which must themselves be given a signature. Hence, this encoding is quite unpractical at large scale. Just compare the one line definition of `hof` in example 6 and the dozen lines of its corresponding implementation on example 7.

[1] One may sometimes work around using the **module type of** construct to derive S' from the inferred type of the functor before packing the functor.

2 MODULAR EXPLICITS

2.1 Presentation

Modular explicits are a solution to the limitation of first-class modules, which they extend by introducing a new type construction in the language, the module-dependent arrow (**module** $\mathcal{X}$: S) $\rightarrow \tau$ where $\mathcal{X}$ may occur free in τ (as the origin of a path leading to an abstract type such as $\mathcal{X}.P.t$). The difference between the module-depend arrow (**module** $\mathcal{X}$: S) $\rightarrow \tau$ and the usual arrow whose domain is a module type (**module** S) $\rightarrow \tau$ is syntactically small, but technically significant: $\mathcal{X} : S$ act as a binder for $\mathcal{X}$ with some limited scope $\mathcal{X}$ which allows $\mathcal{X}$ to appear in (and only in) the codomain type τ. Hence, (**module** $\mathcal{X}$: S) $\rightarrow \tau$ behaves as a first-class polymorphic type[2] while $S \rightarrow \tau$ behaves as a simple type.

Still, when $\mathcal{X}$ does not appear in τ the two forms mean the same and the implementation will actually turn the module-dependent arrow type into an usual arrow type (whose domain is a first-class module type).

The typing of modular explicits is sketched in §A using mini-OCaml language an excerpt of OCaml. As a result of the overlapping of the two kinds of arrows, there is also an overlapping of the typing rules, which is actually the main typechecking issue.

The key property is that when several typing rules applies, they always agree. In fact, we should state it in contraposed form: the typing rules have been designed to prevent the overlapping when the rule would not agree, which crucially relies on the fact that the domain of module-dependent arrow should always be *known* for the type to be treated as a module-dependent arrow, which we have so far left informal.

Indeed, to formalize this concept, we must reveal a detailed that we have hidden so far for sake of simplicity. Arrow types (**module** $\mathcal{X}$: S) $\rightarrow^{\epsilon} \tau$ and $\tau_1 \rightarrow^{\epsilon} \tau_2$ both carry an additional parameter ϵ, called a node-variable, that allows to distinguish between types that are known (when ϵ is generalizable in the current context) and can be treated as dependent arrows and types that are not yet known and should always be treated as non-dependent arrow types. Node variables are of a special kind and incompatible with usual type variables. This mechanism is also necessary to ensure that type inference does not take a decision depending on the order in which type inference and unification constraints are solved, and thus to ensure principal types. See §A.2 for (a few) more details.

Interestingly, the way we distinguish *known* from *guessed* types of module-dependent-functions is similar to a recent proposal by White for adding *semi-explicit polymorphic parameters* [3].

2.2 Compatibility with first class modules

Module-dependent functions are only typing artifacts: their runtime representations are the same as for functions taking first-class modules as arguments. This allows code-free coercions from the type of the former to the type of the latter. In the implementation, the tricky part is the unification of dependent and non-dependent arrow types, which is possible under certain conditions. Namely, (**module** $\mathcal{X}$: S_1) $\rightarrow \tau_1$ and (**module** S_2) $\rightarrow \tau_2$ are unifiable if and only if:

- inlining $\mathcal{X}$ in τ_1 leads to (via type equivalence) a type ${\tau_1}'$ that does not contain a reference to `M` and
- (**module** S_1) $\rightarrow {\tau_1}'$ and (**module** S_2) $\rightarrow \tau_2$.

[2]Module-dependent arrow types are actually no more than first-class polymorphic types and in no way a general form of dependent types. This agrees with the interpretation of modules in F^{ω} [1] and the specific positions of quantifiers in this interpretation.

3 PLAN FOR THE TALK

In the talk, we will motivate the addition of modular explicits along the lines of §1. We will explain modular explicits both from a programmer's point of view, on how to use them, then from the a designer's point of view, showing the smooth interaction with first-class modules, also referring to the key typing rules and internal OCaml type inference mechanism to show how the typing rules are actually implemented.

4 SUMMARY

Modular implicits are a small extension to first-class modules as they do not actually increase expressiveness, but an extension that considerably improves conciseness thanks to a better interaction between the core and module levels, making module-level first-class functors become core-level module-depend functions, and thus enabling some programming with modules directly at the core level without means of encodings.

Although types of module-dependent functions and functions over first-class modules are two different constructions that are typed differently, their overlapping is made mostly transparent to the user, who only sees one kind of arrow that can be treated as a module-dependent arrow type when its domain is *known* and the context *allows* it. Interestingly, the implementation reuses the OCaml existing trick to keep track of principal types and smoothly move from module-dependent arrow types to non-dependent arrow types when needed.

At the time of writing, there is an implementation of modular explicits[3] that is almost ready to be submitted to the OCaml compiler. An experimental version of the compiler with modular explicits can be installed with opam[4]. Technical details, which can be found in appendices, should also serve as a reference specification for this implementation.

Acknowledgment

We would like to thank Vincent Laviron, the Flambda team, and all the other people at OCamlPro for hosting the internship that lead to this implementation. We also thank Leo White for his advices and code reviewing. Some ideas emerged during previous work on modular explicits with Thomas Refis. We wish to acknowledge the work of Matthew Ryan on his own implementation of modular explicits[5], which lead to various discussions on github that gave an insight on their implementation. We also wish to highlight the work of Patrick Reader and Daniel Vlasits under the supervision of Jeremy Yallop as they wrote a significant amount of examples for *modular implicits* that can be used with *modular explicits* when doing the elaboration by hand. Their conclusions also contain a list of features that needs to be done in order to reach a satisfying status for *modular implicits*.

REFERENCES

[1] C. Blaudeau, D. Rémy, and G. Radanne. Fulfilling ocaml modules with transparency. In *Proceedings of the 2024 ACM International Conference on Object Oriented Programming Systems Languages & Applications*, OOPSLA '24, New York, NY, USA, 2024. ACM.

[2] J. Garrigue and D. Rémy. Extending ML with semi-explicit higher-order polymorphism. *Information and Computation*, 155(1/2):134–169, 1999. URL http://www.springerlink.com/content/m303472288241339/. A preliminary version appeared in TACS'97.

[3] L. White. Semi-explicit polymorphic parameters. Presentation at the Higher-order, Typed, Inferred, Strict: ML Family workshops, sep 2023.

[3] Available at https://github.com/samsa1/ocaml/tree/modular-explicit2.

[4] Instructions can be found at https://github.com/samsa1/modular-compiler-variants.

[5] Available at https://github.com/ocaml/ocaml/pull/9187

[4] L. White, F. Bour, and J. Yallop. Modular implicits. In O. Kiselyov and J. Garrigue, editors, *Proceedings ML Family/OCaml Users and Developers workshops, ML/OCaml 2014, Gothenburg, Sweden, September 4-5, 2014.*, volume 198 of *EPTCS*, pages 22–63, 2014. doi: 10.4204/EPTCS.198.2. URL https://doi.org/10.4204/EPTCS.198.2.

$$
\begin{array}{rcll}
P & ::= & \mathcal{X} \mid P.\mathcal{X} \mid P(P) & \text{Path} \\
\tau & ::= & \alpha \mid P.t \mid \tau \to^{\epsilon} \tau \mid (\texttt{module}\ \mathcal{X} : Q) \to^{\epsilon} \tau \mid \ldots & \text{Types} \\
\gamma & ::= & \alpha \mid \epsilon & \text{Type variables} \\
\sigma & ::= & \forall \bar{\alpha}.\, \tau & \text{Type schemes} \\
\rho & ::= & \forall \bar{\epsilon}.\, \sigma & \text{Polytypes} \\
S & ::= & Q \mid \texttt{functor}\ (\mathcal{X} : T) \to S \mid \texttt{sig}\ \bar{D}\ \texttt{end} & \text{Signature} \\
Q & ::= & T \mid P.T \mid Q\ \texttt{with type}\ t = \tau & \text{Named signatures} \\
D & ::= & \texttt{val}\ \ell : \sigma \mid \texttt{type}\ t \mid \texttt{type}\ t = \tau & \\
 & \mid & \texttt{module}\ \mathcal{X} : S \mid \texttt{module type}\ T = S & \text{Specification} \\
a & ::= & x \mid P.\ell \mid \lambda x.\, a \mid a\ a \mid \texttt{let}\ x = a\ \texttt{in}\ a & \text{Expression} \\
 & \mid & (\texttt{module}\ M : Q) \mid \lambda(\texttt{module}\ \mathcal{X} : Q).\, a \mid a\ (\texttt{module}\ M) \ \mid (\sigma) & \\
 & \mid & \texttt{let module}\ \mathcal{X} = M\ \texttt{in}\ a & \text{Explicit expression} \\
M & ::= & P \mid \texttt{struct}\ \bar{d}\ \texttt{end} \mid (M : S) \mid \lambda(\mathcal{X} : S).\, M \mid M\ (\texttt{module}\ M) & \text{Module} \\
d & ::= & \texttt{val}\ \ell = a \mid \texttt{type}\ t \mid \texttt{type}\ t = \tau & \\
 & \mid & \texttt{module}\ \mathcal{X} = M \mid \texttt{module type}\ T = S & \text{Declaration}
\end{array}
$$

Fig. 1. Syntax of mini-OCaml

A SPECIFICATION

A.1 Mini-OCaml

The syntax of (a mini subset of) OCaml is given in Figure 1. The main extension is the introduction of a new form of arrow type $(\texttt{module}\ \mathcal{X} : Q) \to^{\epsilon} \tau$, the module-dependent arrow type. It carries an additional node variable that helps keep track of sharing and ensure the existence of principal types. This is important, but not new, as we are not modifying the usual arrow type $\tau_1 \to \tau_2$, which already contains such an annotation. This which was first introduced [6] for semi-explicit polymorphism [2]. The annotations have been grayed so that you may easily ignore them at first, as OCaml does by default—hence breaking the principal type property. See §A.3.

Notice that we restrict signature annotations at the core-level to named signatures Q, which are a strict subset of signatures S, when used in core language expressions and types. Explicit signatures are (possibly empty) paths to module types, i.e., T or $P.T$, followed by a possibly empty sequence of type constraints $\texttt{with type}\ t = \tau$. This restriction enables to treat first-class module types as named types, which is a restriction enforced by OCaml for simplicity and conciseness of printed types. This applies to all core-level expressions, i.e.,. module-depend arrows and functions and first-class modules, but does not apply for module expressions.

We remind that $\mathcal{X}$ act as a binding in $(\texttt{module}\ \mathcal{X} : Q) \to^{\epsilon} \tau$ whose scope extends to τ and hence may appear in τ. There is an overlap between the two forms of arrows when $\mathcal{X}$ does not actually appear in τ, in which case $(\texttt{module}\ \mathcal{X} : Q) \to^{\epsilon} \tau$ and $(\texttt{module}\ Q) \to^{\epsilon} \tau$ mean the same thing—and the implementation, which treats them as two different type constructors, will freely convert former into the latter in such cases. Here, $(\texttt{module}\ \mathcal{X}) \to^{\epsilon} \tau$ is really a usual arrow $\tau_0 \to^{\epsilon} \tau$ where τ_0 happens to be a first-class module type $(\texttt{module}\ Q)$

We distinguish regular functions $\lambda x.\, a$, which expects an expression as a parameter, from explicit functions $\lambda(\texttt{module}\ \mathcal{X} : Q).\, a$, which expect a module as an argument, whose signature Q must be explicitly given.

[6] This has already been reused for other purposes including for first-class module types to tell when they are known and avoid the need for an annotation on the argument

$$\textsc{Var}\quad \frac{x : \rho \in \Gamma}{\Gamma \vdash x : \rho} \qquad \textsc{App}\quad \frac{\Gamma \vdash a_1 : \tau_2 \to^{\epsilon} \tau_1 \qquad \Gamma \vdash a_2 : \tau_2}{\Gamma \vdash a_1\ a_2 : \tau_1} \qquad \textsc{Fun}\quad \frac{\Gamma, x : \tau' \vdash a : \tau}{\Gamma \vdash \lambda x.\, a : \tau' \to \tau} \qquad \textsc{Gen}\quad \frac{\Gamma \vdash M : \rho \qquad \gamma \# \Gamma}{\Gamma \vdash M : \forall \gamma.\, \rho}$$

$$\textsc{Inst}\quad \frac{\Gamma \vdash M : \forall \alpha.\, \rho}{\Gamma \vdash M : \rho[\alpha \leftarrow \tau]} \qquad \textsc{Inst}\quad \frac{\Gamma \vdash x : \forall \epsilon.\, \rho}{\Gamma \vdash x : \rho} \qquad \textsc{Annot}\quad \Gamma \vdash (\sigma) : \forall \bar{\epsilon}, \bar{\epsilon}', \bar{\alpha}.\, \sigma \to \sigma[\bar{\epsilon} \leftarrow \bar{\epsilon}']$$

$$\textsc{Mod}\quad \frac{\Gamma \vdash M : Q}{\Gamma \vdash (\mathbf{module}\ M : Q) : (\mathbf{module}\ Q)} \qquad \textsc{AppM}\quad \frac{\Gamma \vdash a_1 : (\mathbf{module}\ Q) \to^{\epsilon} \tau \qquad \Gamma \vdash M : Q \qquad \epsilon \# \Gamma}{\Gamma \vdash a_1\ (\mathbf{module}\ M)\ : \tau}$$

$$\textsc{AppD}\quad \frac{\Gamma \vdash a_1 : (\mathbf{module}\ \mathcal{X} : Q) \to^{\epsilon} \tau \qquad \Gamma \vdash M : Q \qquad \Gamma \vdash M \rightsquigarrow P \qquad \epsilon \# \Gamma}{\Gamma \vdash a_1\ (\mathbf{module}\ M)\ : \tau[\mathcal{X} \leftarrow P]}$$

$$\textsc{FunD}\quad \frac{\Gamma, \mathcal{X} : Q \vdash a : \tau}{\Gamma \vdash \lambda(\mathbf{module}\ \mathcal{X} : Q).\, a : (\mathbf{module}\ \mathcal{X} : Q) \to^{\epsilon} \tau}$$

Fig. 2. Typing rules for functions and applications

Similarly, we distinguish the regular application $a_1\ a_2$ from the application to an immediate module expression $a_1\ (\mathbf{module}\ M)$. While we may also turn a module into a first-class module expression $(\mathbf{module}\ M : Q)$, which can then be stored in a data-structure or passed to functions, the immediate applications $a\ (\mathbf{module}\ M)$ does not require an explicit signature Q for M when the type of a is known.

A.2 Typing rules

An excerpt of typing rules for expressions is given on Figure 2. The judgment $\Gamma \vdash M : Q$ for typing modules is not affected by this extension and is left implicit.

The main rule is the typing of modular-explicits functions $\lambda(\mathbf{module}\ \mathcal{X} : Q).\, a$, i.e., core-language functions expecting a module as argument.

We have three rules for typing applications.

- A dependent application AppD, can only be used when the type is known ($\epsilon \# \Gamma$) to be a dependent function type $(\mathbf{module}\ \mathcal{X} : Q) \to^{\epsilon} \tau$ and the argument of the application is an immediate first-class module $(\mathbf{module}\ M)$ where M is of type Q can be elaborated to a path P.
- A non-dependent application AppM can be used when the type is known ($\epsilon \# \Gamma$) to be $(\mathbf{module}\ Q) \to^{\epsilon} \tau$ and the argument of the application is an immediate first-class module $(\mathbf{module}\ M)$ where M is of type Q.
- Otherwise, we default to a normal application App of an expression a_1 to an expression a_2. Hence a_1 is a non dependent arrow whose domain may be unknown (ϵ need not be polymorphic).

Rule Mod turns a module M into a first-class expression. It requires an explicit signature Q.

As a particular case of Rule App, a_2 may still be a first-class module $(\mathbf{module}\ M : Q)$, but it has be to explicitly annotated with a signature Q, since the Q is not known from context. That is, even if τ_2 happens to be $(\mathbf{module}\ Q)$, this information may depend on the order in which type inference and type constraint will have been solved, and we therefore refuse to take advantage of the knowledge

of τ_2 to elude the type annotation. This would just the following instance of App:

$$\frac{\begin{array}{c}\textsc{App}\\ \Gamma \vdash a_1 : (\mathbf{module}\ S) \rightarrow^{\epsilon} \tau_1 \qquad \Gamma \vdash a_2 : (\mathbf{module}\ Q)\end{array}}{\Gamma \vdash a_1\ a_2}$$

Rules AppD and AppM are not exclusive in the case where $\mathcal{X} \notin \tau$. In that case, both rules return the same type because $\tau[\mathcal{X} \leftarrow P]$ is then equal to τ, hence it does not break principality.

A.3 On principality

Typing rules AppD and AppM for applications both require the type of a to be *known*, which is ensured by the side condition $\epsilon \# \Gamma$. (Equivalently, we could also have written $\Gamma \vdash a_1 : \forall\epsilon.\ (\mathbf{module}\ \mathcal{X} : Q) \rightarrow^{\epsilon} \tau$, since this could be inferred from $\epsilon \# \Gamma$ by Rule Gen—and conversely, using rule Inst. These side conditions still allow to guess types, but prevent the use of *guessed* types as *known* types. For instance, an attempt typing

$$\lambda f.\ a[f\ (\mathbf{module}\ M)]$$

will fail, as as if we guessed that f had the module-dependent type $(\mathbf{module}\ \mathcal{X} : S) \rightarrow^{\epsilon} \tau$[7], then ϵ would appear in the typing context while typing the expression $f\ (\mathbf{module}\ M)$ and rule AppD would not apply. The key is that guessing an arrow type τ that appears in Γ makes ϵ appear in τ and thus also in Γ.

By contrast, FunD introduces a node variable ϵ in the type $(\mathbf{module}\ \mathcal{X} : Q) \rightarrow^{\epsilon} \tau$ of its conclusion, which may be taken fresh for Γ, thus letting the (module-depndent) arrow type be considered as *known*. Typically, in an expression of the form

$$\mathbf{let}\ f = \lambda(\mathbf{module}\ \mathcal{X} : Q).\ a_0\ \mathbf{in}\ a[f\ (\mathbf{module}\ M)]$$

variable ϵ will be generalized in the type of x so that the application $x\ (\mathbf{module}\ M)$ can be typed without the need for an annotation on M.

A guessed type may also be turned into a known type using a *type annotation* $M : \sigma$, which is syntactic sugar for $(\sigma)\ a$, i.e., the application of the retyping function (σ) to a, which just changes the type of a. Rule Annot, gives σ the principal type $\forall\bar{\epsilon}, \bar{\epsilon}', \bar{\alpha}.\ \sigma \rightarrow \sigma[\bar{\epsilon} \leftarrow \bar{\epsilon}']$ where $\bar{\alpha}$ is the set of all free type variables of σ. The effect (σ) is to return a clone of the type of its argument after renaming of node-variables to fresh ones.

We also provide the syntactic sugar $\lambda(x : \sigma).\ a$ for $\lambda x.\ \mathbf{let}\ x = (x : \sigma)\ \mathbf{in}\ a$. This tells that the parameter x is an instance τ of σ, whose the polymorphic skeleton, σ, is known. This allows to type

$$\lambda f : (\mathbf{module}\ \mathcal{X} : S) \rightarrow^{\epsilon} \tau.\ a[f\ (\mathbf{module}\ M)]$$

without an annotation on M, since f may be assigned the polytype $\forall\epsilon.\ (\mathbf{module}\ \mathcal{X} : S) \rightarrow^{\epsilon} \tau$ while the expression $f\ (\mathbf{module}\ M)$. in the body a. Conversely, not only $\lambda f.\ a[f\ (\mathbf{module}\ M)]$ will fail, as explained above, but also

$$\lambda x.\ \lambda f : (\mathbf{module}\ \mathcal{X} : S) \rightarrow^{\epsilon} \tau.\ \mathbf{let}\ g = \mathit{either}\ x\ f\ \mathbf{in}\ a[g\ (\mathbf{module}\ M)]$$

Indeed, the type of g which mixes the known type of f with the guessed type of x becomes a guessed type.

Thanks to these side conditions on node-variables, the type system has the principal type property, i.e., typable expressions always have principal types. While omitting those side conditions could break principality—it would still preserve type safety. Indeed, the OCaml ignores the side condition by default—unless explicitly told otherwise as explained below.

[7]During type inference, this would amount to subtituting the unknown type α of f that appears in the typing context by $(\mathbf{module}\ \mathcal{X} : S) \rightarrow^{\epsilon} \tau$.

A.4 Implementation details

The implementation of modular explicits contains a few subtleties that should be reported:

- Current type inference in OCaml typechecks n-ary applications $\overline{\tau_i \rightarrow} \tau$ by pairing each argument with its expected type before typechecking the arguments. However when typing a dependent function application, we typecheck the module before looking at the rest of arguments.
- Unification in OCaml links both types together. Thus, when unifying $(\textbf{module}\ \mathcal{X}_1 : Q_1) \rightarrow^{\epsilon} \tau_1$ with $(\textbf{module}\ \mathcal{X}_2 : Q_2) \rightarrow^{\epsilon} \tau_2$ we need to unify τ_1 and τ_2 together, leaving us a type τ which is a mix of both. However τ_1 and τ_2 did not live in the same context. Thus we also unify $\mathcal{X}_1$ with $\mathcal{X}_2$ in order to ensure the validity of $(\textbf{module}\ \mathcal{X}_1 : Q_1) \rightarrow^{\epsilon} \tau$ and $(\textbf{module}\ \mathcal{X}_2 : Q_2) \rightarrow^{\epsilon} \tau$.
- The OCaml typechecker is order dependent, i.e., some programs are accepted based on the order of in which type inference proceeds. Modular explicits are also impacted by this. We use the same mechanism as with labels or first-class modules to allow the user to receive warnings (which can be turned into errors that reject the program) when non-principal types are inferred, i.e., using the *-principal* option.
  By default, OCaml may already infer non-principal types as this allows removing some type annotations—and avoids duplicating types at some program points with could significantly slow down type checking.
- As with first-class modules, the arguments of two module-dependent functions are compatible only if both modules have the same runtime representation and are coercible to one another, i.e., the values in the same order but types can change position. By contrast, ground coercion requires only the same runtime representation and one way of coercion.

B CODE EXAMPLES

B.1 Dependent function as functions other first-class modules

Let us assume the following preliminary definition

```
module type S = sig type t end
```

Then, example 1 below fails to typecheck because `int` is not compatible with `bool`.

```
let app (f : (module S with type t = 'a) -> 'a -> 'a) =
    (f (module Int) 3, f (module Bool) true)
```

Example 1. Ill-typed naive use of first-class modules

Rewriting the example with module-dependent function, as in example 2, typechecks:

```
let app (f : (module M : S) -> M.t -> M.t) =
    (f (module Int) 3, f (module Bool) true)
```

Example 2. Using module dependant function

A work around using first-class modules is to enforce polymorphism with some wrapper as in example 3 or example 4. The difference with example 1 is that we only loose polymorphism when accessing inside the wrapper and we can do this at each call-site.

```
let app (f : <m : 'a. (module S with type t = 'a) -> 'a -> 'a >) =
    (f#m (module Int) 3, f#m (module Bool) true)
```

Example 3. Example using an object as a wrapper

```
module type Fwrapper = sig
     val f : (module S with type t = 'a) -> 'a -> 'a
end

let app (module F : Fwrapper) = (F.f (module Int) 3, F.f (module Bool) true)
```

Example 4. Using first-class module as a wrapper

B.2 Dependent function as functions as first-class functors

Example 7 illustrates the encoding of module-dependent functions as first-class functors that works well with higher-order abstract types. The conciseness and clarity of the direct use module-dependent functions should be striking.

```
module type S = sig
  type _ t
  val f : 'a -> 'a t
end

module List = struct
  type 'a t = 'a list
  let f x : _ t = [x]
end
```

Example 5. The shared signature and module

```
let hof x (module M : S) = M.f x
: 'a -> (module M : S) -> 'a M.t

let hoapp = hof 3 (module List)
```

Example 6. With a module dependent function

```
module type HOF = sig
  type t
  module F (M : S) :
    sig val value : t M.t end
end

let hof (type a) (x : a) =
  let module Hof = struct
    type t = a
    module F (M : S) =
      struct let value = M.f x end
  end in
  (module Hof : HOF with type t = a)
: 'a -> (module HOF with type t = 'a)

let hoapp =
  let module Hof = (val (hof 3)) in
  let module H = Hof.F(List) in H.value
```

Example 7. Without module dependent function

Fig. 3. Example with a parametric abstract type